ARDUINO DRIVER // TECHNICAL REFERENCE

ST7735 Arduino Driver

Полная документация по API, виджетам и оптимизациям

RGB565-драйвер для TFT-дисплеев 1.77"/1.8" 160×128 с поддержкой AVR, ESP32 и STM32. Безмерцательный вывод, инкрементальный рендер Plotter, аппаратный скролл терминала, виджеты Value / BarGauge / Plotter / Log / Screens и кириллица UTF-8 → CP1251 с белорусскими буквами Ўў Іі.

АВТОР

AlecsHome

V5.0 AVR ESP32 STM32

Содержание

1. Введение и обзор **4**

1.1. Что такое драйвер ST7735 . . . 4

1.2. Возможности . . . 4

1.3. Поддерживаемые платформы . . . 5

1.4. Архитектура библиотеки . . . 5

1.5. Ключевые оптимизации . . . 6

2. Установка **6**

2.1. Копирование файлов . . . 6

2.2. Структура библиотеки . . . 6

2.3. Перезапуск Arduino IDE . . . 7

3. Подключение пинов **7**

3.1. Arduino Uno / Nano / Pro Mini (ATmega328) . . . 7

Важные замечания по питанию и SS . . . 8

3.2. Подключение SD-карты и других SPI-устройств . . . 8

3.3. Подключение на Arduino Mega, ESP32, STM32 . . . 8

3.4. Минимальный скетч — проверка работоспособности . . . 8

4. Конфигурация (макросы перед #include) **9**

4.1. Полный список макросов . . . 10

4.2. Принцип «включай только нужное» . . . 10

4.3. Пример включения виджетов . . . 11

5. API класса ST7735 **11**

5.1. Конструктор и инициализация . . . 12

begin() — инициализация дисплея . . . 12

5.2. Ориентация . . . 12

5.3. SPI-сессии: startWrite / endWrite . . . 13

5.4. Низкоуровневые методы . . . 13

5.5. Графические примитивы . . . 14

5.6. Окружности, скруглённые прямоугольники, эллипсы . . . 14

5.7. Треугольники . . . 15
5.8. Изображения RGB565 и 1-битные битмапы . . . 15
5.9. drawProgressBar — дифференциальная перерисовка . . . 15
5.10. Управление дисплеем . . . 16
5.11. Аппаратный скролл . . . 16
5.12. Текст: шрифты, курсор, цвета, масштаб . . . 16
clearLine / clearLineBg — очистка строки . . . 17
5.13. Вывод: print, println, drawString, drawChar . . . 17
5.14. Терминал (только при ST7735_ENABLE_TERMINAL=1) . . . 18
Аппаратный скролл терминала . . . 19
5.15. printf (опционально) . . . 19
6. UI-виджеты (st7735ui.h) 19
6.1. ST7735Button — кнопка с press-состоянием . . . 19
6.2. ST7735Toggle — тумблер ON/OFF . . . 20
6.3. ST7735Checkbox — чекбокс с подписью . . . 21
6.4. ST7735Value — числовой индикатор (главный виджет) . . . 21
Skip-redraw оптимизация . . . 22
Авто-масштаб . . . 22
Состояния виджета . . . 22
6.5. ST7735BarGauge — шкальный индикатор с порогом . . . 23
6.6. ST7735Plotter — осциллограф/график реального времени . . . 23
Multi-strip рендер . . . 24
Инкрементальный рендер . . . 24
Оверлей ВЫНЕСЕН из класса . . . 24
6.7. ST7735Log — журнал сообщений . . . 25
6.8. ST7735Screens — переключатель экранов . . . 26
7. Шрифты 26
7.1. font5x7 и font7x11 . . . 27
7.2. Дескриптор шрифта . . . 27
7.3. Кириллица UTF-8 → CP1251 . . . 27
7.4. Формулы размеров текста . . . 28
7.5. Памятка: сколько символов влезает . . . 28

8. Цвета (color565.h) 28

8.1. Формат RGB565 . . . 28

8.2. Предопределённые константы . . . 29

9. Оптимизации 29

9.1. Таблица оптимизаций . . . 30

9.2. Принцип безмерцательного вывода . . . 30

9.3. Стратегия: batch через startWrite/endWrite . . . 31

9.4. Кэш окна setAddrWindow . . . 31

10. Платформы 31

10.1. AVR (ATmega328PB / MiniCore) . . . 31

10.2. ESP32 / ESP8266 . . . 31

10.3. STM32duino . . . 31

10.4. STM32 HAL (CubeMX) . . . 32

11. Примеры 32

11.1. Hello.ino — минимальная проверка . . . 32

11.2. Volt.ino — ST7735Value . . . 33

11.3. BarGauge.ino — ST7735BarGauge . . . 34

11.4. Plotter.ino — ST7735Plotter с внешним оверлеем . . . 35

11.5. UI_Demo.ino — полный демо . . . 36

12. Troubleshooting / FAQ 37

13. Лицензия и контакты 39

13.1. Лицензия . . . 39

13.2. Контакты и исходный код . . . 39

1. Введение и обзор

1.1. Что такое драйвер ST7735

ST7735 Arduino Driver — компактный и платформенно-независимый драйвер для TFT-дисплеев на контроллере ST7735R/ST7735S с разрешением 160×128 пикселей (1.77" и 1.8" модули). Драйвер написан на C++ и рассчитан на использование из Arduino IDE, а также на «голом» STM32 через HAL. В отличие от классических библиотек Adafruit_ST7735 + Adafruit_GFX, данный драйвер изначально спроектирован для ресурсо-ограниченных платформ: каждый модуль можно выключить макросом, а все «тяжёлые» операции (float-форматирование, отрисовка графиков, скролл терминала) реализованы без зависимостей от libprintf_flt, dtostrf и math.h.

Драйвер покрывает три уровня абстракции: низкоуровневый обмен с контроллером по SPI, графические примитивы (точки, линии, прямоугольники, окружности, эллипсы, треугольники, изображения RGB565 и 1-битные битмапы), а также модуль текста с поддержкой кириллицы (UTF-8 → CP1251) и терминала с автоскроллом. Сверху на ядро ST7735 надстроен модуль st7735ui.h с восьмью готовыми UI-виджетами: кнопка, тумблер, чекбокс, числовой индикатор, шкальный индикатор, осциллограф-плоттер, журнал сообщений и переключатель экранов.

1.2. Возможности

- Цветовой режим RGB565: 5 бит R, 6 бит G, 5 бит B — 16 бит/пиксель, как в GRAM контроллера.
- Поддержка табов ST7735R_RED / GREEN / GREEN3 / BLACK — выбор по цвету плёнки дисплея.
- Безмерцательный вывод: каждый пиксель пишется ровно один раз за кадр (нет «залил-стёр-нарисовал»).
- Инкрементальный рендер Plotter: перерисовываются только изменившиеся колонки, экономия SPI до 95%.
- Аппаратный скролл терминала через команды VSCRDEF/VSCSAD — прокрутка строк без CPU-копирования GRAM.
- Шрифты font5x7 (5×7) и font7x11 (7×11) с масштабированием setTextScale(1..N).
- Кириллица UTF-8 → CP1251, включая белорусские буквы Ў ў І і — вывод без отдельных шрифтовых файлов.
- Дифференциальная перерисовка виджетов: ST7735Value skip-redraw при том же отформатированном значении.
- Чистый C++ без dtostrf / snprintf / math.h — компилируется на AVR, ESP32, STM32duino, STM32 HAL.
- Опциональный мост Adafruit_ST7735_GFX.h для совместимости со старым кодом через Adafruit_GFX API.

1.3. Поддерживаемые платформы

Драйвер проверен на четырёх семействах платформ. На каждой из них доступен один и тот же API; разница — только в производительности и объёме доступной памяти, что влияет на рекомендуемые значения макросов конфигурации.

Платформа	Среда	Особенности
AVR (ATmega328PB / MiniCore)	Arduino IDE	2 КБ RAM, 32 КБ flash. double=float. CHUNK=64, INCREMENTAL выключен по умолчанию.
ESP32 / ESP8266	Arduino IDE	≥ 320 КБ RAM. Все макросы можно включить. CHUNK=256, INCREMENTAL=1.
STM32duino (BluePill, Maple)	Arduino IDE	≥ 20 КБ RAM. Аналогично ESP32 — все макросы включаются свободно.
STM32 HAL (CubeMX)	Bare C/C++	Через макрос ST7735_BARE_STM32 + ST7735_CS_GPIO/PIN и т.д. SPI через HAL_SPI_Transmit.

1.4. Архитектура библиотеки

Библиотека разделена на три уровня. Ядро st7735.h / st7735.cpp содержит класс ST7735 с примитивами, текстом и (опционально) терминалом. Модуль st7735ui.h содержит UI-виджеты, каждый из которых компилируется только если соответствующий макрос ST7735_ENABLE_* задан в 1. Файл Adafruit_ST7735_GFX.h — опциональный мост, который адаптирует API ядра под Adafruit_GFX для совместимости со сторонним кодом; ядро его никогда не требует.

Файл	Слой	Назначение
st7735.h, st7735.cpp	Ядро	Класс ST7735: SPI, примитивы, текст, терминал, скролл.
color565.h	Ядро	RGB565 константы цветов и макрос RGB565(R,G,B).
font5x7.h, font7x11.h	Ядро	Дескрипторы шрифтов и таблицы глифов (CP1251).
st7735ui.h	UI-слой	ST7735Button / Toggle / Checkbox / Value / BarGauge / Plotter / Log / Screens.
Adafruit_ST7735_GFX.h	Мост	Адаптация API под Adafruit_GFX (включается макросом ST7735_ENABLE_Adafruit).
keywords.txt	IDE	Подсветка ключевых слов в Arduino IDE.
examples/	Демо	Hello, Volt, BarGauge, Plotter, UI_Demo — рабочие скетчи.

1.5. Ключевые оптимизации

Драйвер целенаправленно оптимизирован под сценарий реального времени: опрос датчиков с быстрым обновлением экрана без мерцания. В основе всех оптимизаций лежит принцип «один пиксель — одна запись в GRAM»: ни одна операция не использует паттерн «залить фон → стереть → нарисовать». Вместо этого виджеты предварительно вычисляют изменившуюся зону и пишут только её, а дисперсные операции (риски шкалы, оверлеи графиков) встраиваются в общий SPI-поток через startWrite() / endWrite().

Оптимизация	Эффект
Plotter инкрементальный рендер	до 95% экономии SPI для медленных сигналов (не-AVR)
ST7735Value skip-redraw	до 99% экономии для polling-датчиков (по отформатированной строке)
Терминал HW scroll (VSCRDEF/VSCSAD)	~95% экономии при лог-выводе, без CPU-копирования строк
drawLine с Liang-Barsky clips	нет 65 000 итераций Bresenham для диагоналей вне экрана
Multi-strip Plotter	график шире ST7735_PLOTTER_MAX_W не обрезается, рисуется несколькими полосами
memmove в termScroll	убран UB, в десятки раз быстрее при больших размерах терминала
st7735_fmtDouble epsilon	защита IEEE-754 неточности (нет «:» вместо цифры)

2. Установка

2.1. Копирование файлов

Драйвер не распространяется через Arduino Library Manager. Установка — ручная: скопируйте все файлы библиотеки в директорию Documents/Arduino/libraries/ST7735/ (Windows: Мои документы\Arduino\libraries\ST7735). После копирования полностью перезапустите Arduino IDE, чтобы среда пересканировала библиотеки. Если всё прошло успешно, в меню Файл → Примеры появится раздел «ST7735» с пятью скетчами: Hello, Volt, BarGauge, Plotter, UI_Demo.

2.2. Структура библиотеки

Файлы библиотеки должны лежать плоско, без вложенных папок (кроме examples/). Поддерживаются и .h-исходники в корне, и папка utility/ — но в текущей поставке используется плоская схема.

```
libraries/ST7735/
├── st7735.h ← ядро: класс ST7735
├── st7735.cpp ← реализация ядра
├── st7735ui.h ← UI-виджеты (header-only)
├── color565.h ← RGB565 константы
├── font5x7.h ← шрифт 5×7 + таблица CP1251
├── font7x11.h ← шрифт 7×11 + таблица CP1251
├── Adafruit_ST7735_GFX.h ← опциональный мост к Adafruit_GFX
├── keywords.txt ← подсветка API в Arduino IDE
├── README.md ← краткое описание
└── examples/
├── Hello/hello.ino
├── Volt/volt.ino
├── BarGauge/bargauge.ino
├── Plotter/plotter.ino
└── UI_Demo/ui_demo.ino
```

2.3. Перезапуск Arduino IDE

Arduino IDE кэширует список библиотек при запуске. После копирования файлов обязательно полностью закройте IDE (включая все открытые окна) и запустите её заново — иначе новые ключевые слова не подсветятся, а примеры не появятся в меню. Если в скетче возникает ошибка 'ST7735' does not name a type, проверьте, что папка libraries/ST7735/ лежит именно в пользовательской папке Arduino, а не в папке установки IDE.

3. Подключение пинов

3.1. Arduino Uno / Nano / Pro Mini (ATmega328)

На AVR аппаратный SPI физически закреплён за ножками чипа, поэтому в конструкторе ST7735 передаются только управляющие пины дисплея: CS, DC, RST. Пины MOSI/SCK/MISO библиотека использует через SPI.h — их в конструкторе указывать не нужно.

Пин Arduino	Сигнал	Назначение
10	TFT_CS	D10 — SS AVR; держать OUTPUT всегда, иначе SPI сломается.
9	TFT_DC	Data/Command — выбор режима (команда или данные GRAM).
8	TFT_RST	Аппаратный сброс; можно -1 если RST на плате дисплея к VCC.
11	MOSI	Аппаратный SPI MOSI — к TFT SDA / DIN.
13	SCK	Аппаратный SPI SCK — к TFT SCK / CLK.
12	MISO	Аппаратный SPI MISO; если не используется SD — pinMode INPUT.

Важные замечания по питанию и SS

ST7735 питается от 3.3 В. Подача 5 В на VCC дисплея приводит к выходу из строя контроллера или к искажённым цветам. Если плата Arduino — 5-вольтовая (Uno, Nano, Pro Mini 5V), ставьте преобразователь уровня на линиях MOSI/SCK/CS/DC/RST либо используйте платы с нативным 3.3 В (Pro Mini 3.3V, ESP32, STM32). На пин 10 (SS AVR) — особое требование: даже если вы используете его как CS дисплея, он обязан всегда быть сконфигурирован как OUTPUT. Если SS окажется INPUT и плавает, SPI переведётся в slave-режим и связь с дисплеем оборвётся.

3.2. Подключение SD-карты и других SPI-устройств

Несколько SPI-устройств (дисплей + SD-карта + датчик) подключаются к общим линиям MOSI/SCK/MISO и имеют уникальные пины CS. Перед обращением к устройству его CS опускается в LOW, после — поднимается в HIGH. Библиотека ST7735 автоматически управляет своим CS через startWrite() / endWrite() и не требует ручного digitalWrite.

Устройство	CS	Замечание
SD-карта (MicroSD breakout)	CS на пин 4 (Arduino)	общие MOSI=11 / SCK=13 / MISO=12.
Датчик с SPI (например MCP3208)	CS на любой свободный пин	общие MOSI/SCK/MISO.
Второй ST7735 (мало вероятно)	CS на свой пин	DC и RST можно разделить или разделять.

3.3. Подключение на Arduino Mega, ESP32, STM32

Платформа	Управляющие пины	SPI пины / замечание
Arduino Mega 2560	CS=53, DC=9, RST=8	SPI: MOSI=51, SCK=52, MISO=50 (HSPI).
ESP32	CS=GPIO5, DC=GPIO16, RST=GPIO4	SPI: MOSI=23, SCK=18, MISO=19 (VSPI). Либо HSPI: 13/14/12.
STM32duino (BluePill)	CS=PA4, DC=PA3, RST=PA2	SPI1: MOSI=PA7, SCK=PA5, MISO=PA6.
STM32 HAL (CubeMX)	через ST7735_CS_GPIO / ST7735_CS_PIN	Только макросы; пины задаются в -D или main.h.

3.4. Минимальный скетч — проверка работоспособности

Ниже — минимальный пример, выводящий на дисплей жёлтый крупный текст «Hello!» и подтверждает, что драйвер установлен, пины распаяны правильно, а таб выбран верно. Полный пример Hello.ino демонстрирует также примитивы (drawRect, fillCircle, drawEllipse, drawLine) и кириллицу.

cpp — hello_minimal.ino

```cpp
#include "st7735.h"

ST7735 tft(10, 9, 8, ST7735R_BLACK);

void setup() {
tft.begin(); /* SPI 8 МГц по умолчанию */
tft.fillScreen(ST7735_BLACK);

tft.setTextColor(ST7735_YELLOW, ST7735_BLACK);
tft.setFont(&font7x11;);
tft.setTextScale(2);
tft.setCursor(5, 5);
tft.print("Hello!");
}

void loop() {}
```

4. Конфигурация (макросы перед #include)

Все макросы конфигурации задаются ДО #include "st7735.h" (или st7735ui.h). Если макрос уже определён к моменту #include, библиотека его уважает; иначе подставляется значение по умолчанию. Дефолтное значение 0 для всех модулей — это сознательный «breaking change»: каждый скетч явно декларирует свои зависимости, и никто не тянет в бинарник мусор. При обновлении библиотеки старые скетчи потребуют правки — но это намного лучше, чем тихо раздувающийся flash на AVR.

4.1. Полный список макросов

Макрос	Дефолт	Описание
ST7735_ENABLE_TERMINAL	0	Терминальный модуль (setTerminal/termClear/print-в-область).
ST7735_ENABLE_Adafruit	0	Мост к Adafruit_GFX. Ядро его не проверяет — только Adafruit_ST7735_GFX.h.
ST7735_ENABLE_PRINTF	0	Метод printf(fmt, ...). Тянет vsnprintf из libc (~200 Б + 1–2 КБ).
ST7735_ENABLE_ARC	0	fillArc() — кольцевой сектор. Тянет sinf/cosf из libm (~1 КБ).
ST7735_ENABLE_BUTTON	0	Класс ST7735Button (~0.3 КБ flash).
ST7735_ENABLE_TOGGLE	0	Класс ST7735Toggle (~0.3 КБ flash).
ST7735_ENABLE_CHECKBOX	0	Класс ST7735Checkbox (~0.4 КБ flash).
ST7735_ENABLE_VALUE	0	Класс ST7735Value (~1.2 КБ flash, авто-масштаб).
ST7735_ENABLE_BARGAUGE	0	Класс ST7735BarGauge (~0.8 КБ flash, дифф-перерисовка).
ST7735_ENABLE_PLOTTER	0	Класс ST7735Plotter (~1.5 КБ flash, авто-диапазон).
ST7735_ENABLE_LOG	0	Класс ST7735Log (кольцо из ST7735_LOG_LINES×COLS).
ST7735_ENABLE_SCREENS	0	Класс ST7735Screens (циклический переключатель экранов).
ST7735_TEXT_MAX_PX	28	Макс. ширина глифа, px (_charBuf = 2×MAX_PX байт RAM).
ST7735_TEXT_MAX_H	16	Макс. высота шрифта (bpc=2 → до 16 строк пикселей).
ST7735_TERM_ROWS	16	Строки терминала (RAM = ROWS×COLS байт).
ST7735_TERM_COLS	27	Столбцы терминала.
ST7735_CHUNK	64/256	Размер SPI-буфера. AVR=64 (128 Б стек), не-AVR=256 (512 Б).
ST7735_PLOTTER_MAX_PTS	120	Макс. точек графика (uint8_t — верхний предел 255).
ST7735_PLOTTER_MAX_W	160	Ширина полосы рендера (static lo[]/hi[]). Для 320×240 — 320.
ST7735_LOG_LINES	8	Строк журнала ST7735Log.
ST7735_LOG_COLS	30	Символов в строке журнала.
ST7735_MAX_SCREENS	4	Макс. экранов в ST7735Screens.

4.2. Принцип «включай только нужное»

На AVR (2 КБ RAM, 32 КБ flash) каждый включённый модуль заметен в размере бинарника. Стандартный приём — составить список реально используемых виджетов и включить только их. На ESP32 / STM32 / STM32 HAL можно включить все макросы безболезненно — запас памяти позволяет.

Ориентировочная экономия flash на ATmega328PB при выключении модуля приведена в таблице ниже. Цены — относительно конфигурации «всё включено».

Макрос	Экономия flash	Что именно экономится
ST7735_ENABLE_PLOTTER	~1.5 КБ	Парсер, redrawGraph, авто-диапазон.
ST7735_ENABLE_VALUE	~1.2 КБ	st7735_fmtDouble (общая часть — в st7735.h), авто-масштаб.
ST7735_ENABLE_BARGAUGE	~0.8 КБ	drawTicks + дифф-перерисовка.
ST7735_ENABLE_CHECKBOX	~0.4 КБ	Рисование галочки.
ST7735_ENABLE_TOGGLE	~0.3 КБ	fillRoundRect + fillCircle.
ST7735_ENABLE_BUTTON	~0.3 КБ	fillRoundRect + drawStringAligned.
ST7735_ENABLE_TERMINAL	~1.5 КБ + 444 Б RAM	_termBuf = ROWS×COLS байт.
ST7735_ENABLE_PRINTF	~200 Б + vsnprintf	vsnprintf из libc тянет ещё 1–2 КБ.
ST7735_ENABLE_ARC	~1 КБ	sinf/cosf из libm.

4.3. Пример включения виджетов

Порядок важен: сначала #include "st7735.h" (чтобы макросы ядра применились), потом #define-ы для UI-модулей, потом #include "st7735ui.h". Если задать макросы после include — библиотека использует дефолты.

cpp — выборочное включение UI

```cpp
#include "st7735.h"

/* Включаем только нужные виджеты */
#define ST7735_ENABLE_VALUE 1
#define ST7735_ENABLE_BARGAUGE 1
#define ST7735_ENABLE_PLOTTER 1
/* Button и Toggle остаются выключенными — их код не попадёт в бинарник */

#include "st7735ui.h"
```

Если в коде используется выключенный класс, компилятор выдаст ясную ошибку вида: «'ST7735Value' does not name a type». Это подсказка: добавьте соответствующий #define ST7735_ENABLE_VALUE 1 перед #include "st7735ui.h".

5. API класса ST7735

В этой главе — детальный справочник по всем публичным методам класса ST7735. Методы сгруппированы по функциональным областям: конструктор и инициализация, ориентация, SPI-сессии, низкоуровневый обмен, графические примитивы, изображения, текст, терминал. Для каждого метода приведены сигнатура, описание параметров и краткий пример вызова.

5.1. Конструктор и инициализация

Конструктор ST7735 принимает три управляющих пина и таб — идентификатор варианта платы дисплея. На AVR пины SPI в конструкторе не указываются (они физически закреплены).

```cpp — конструктор
ST7735(int8_t cs = -1, int8_t dc = -1, int8_t rst = -1,
uint8_t tab = ST7735R_BLACK);
```

Параметр tab выбирается по цвету плёнки на стекле модуля. Если цвета на экране инвертированы или изображение сдвинуто — поменяйте таб.

Таб	Знач.	Описание
ST7735R_RED	0	Красная плёнка: смещения (0,0), BGR, без инверсии.
ST7735R_GREEN	1	Зелёная: смещения (2,1).
ST7735R_GREEN3	2	Зелёная поздняя: (2,1) + INVON.
ST7735R_BLACK	3	Чёрная: (0,0), RGB — типично для 1.77" 160×128.

begin() — инициализация дисплея

```cpp
void begin(uint32_t spiHz = 8000000UL);
```

Инициализирует SPI, аппаратно сбрасывает дисплей, отправляет инициализационную последовательность для выбранного таба, включает дисплей (DISPON) и выходит из спящего режима (SLPOUT). По умолчанию SPI работает на 8 МГц — это безопасный максимум для ST7735. На AVR бóльшие частоты (например 16 МГц через SPI_CLOCK_DIV2) на длинных проводах могут давать сбои; поднимайте частоту только на коротких проводниках. Если передать 0 или очень низкое значение (например 9600 Гц) — экран будет «висеть», не путайте с Serial.begin(9600).

5.2. Ориентация

```cpp
void setRotation(uint8_t r); /* 0..3 */
uint8_t getRotation() const; /* вернуть текущий */
int16_t width() const; /* _width после поворота */
int16_t height() const; /* _height после поворота */
void setOffsets(uint8_t colstart, uint8_t rowstart);
```

setRotation(0..3) поворачивает изображение на 0°, 90°, 180°, 270° через биты MY/MX/MV регистра MADCTL. После поворота width()/height() меняются местами для 90°/270°. Для большинства 1.77" модулей портрет (rotation=0) даёт 128×160, ландшафт (rotation=1) — 160×128. Смещения colstart/rowstart обычно задаются автоматически из таба, но setOffsets позволяет их переопределить — полезно для нестандартных модулей со смещённым GRAM.

5.3. SPI-сессии: startWrite / endWrite

```cpp
void startWrite(); /* опустить CS, пометить «в сессии» */
void endWrite(); /* поднять CS, пометить «сессия закрыта» */
```

Каждая операция с дисплеем — это SPI-транзакция. Чтобы не дёргать CS на каждый байт, библиотека группирует операции в сессии. startWrite() опускает CS в LOW и инкрементирует внутренний счётчик _inWrite; endWrite() декрементирует его и поднимает CS только когда счётчик дошёл до 0. Сессии вкладываются: если вы вызываете startWrite() внутри другой сессии, CS останется LOW до закрытия всех уровней.

Вложенные сессии важны для виджетов: ST7735Button::draw() открывает свою сессию и внутри неё вызывает fillRoundRect / drawRoundRect / drawStringAligned, каждый из которых открывает свою startWrite(). Без счётчика CS поднимался бы после каждой подоперации и SPI простаивал бы между ними. Все публичные методы ядра (drawPixel, drawLine, fillRect, print, …) сами управляют своими сессиями — ручной startWrite нужен только если вы батчите несколько вызовов сами или вызываете низкоуровневые writePixels/writeColor.

5.4. Низкоуровневые методы

Эти методы вызываются между startWrite() / endWrite(). Они НЕ управляют CS самостоятельно — это ответственность вызывающего. Снаружи startWrite/endWrite их использовать нельзя: CS будет плавать и SPI сломается.

```cpp — низкий уровень
void writeCommand(uint8_t c); /* DC=0, отправить байт команды */
void writeData8(uint8_t d); /* DC=1, отправить байт данных */
void writeData16(uint16_t d); /* DC=1, старший байт вперёд */
void setAddrWindow(int16_t x0, int16_t y0,
int16_t x1, int16_t y1); /* CASET+RASET+RAMWR */
void writePixels(const uint16_t *colors, uint32_t count);
void writeColor(uint16_t color, uint32_t count);
```

setAddrWindow() открывает прямоугольную область в GRAM контроллера (команды CASET, RASET, RAMWR) — после неё можно отправлять пиксели потоком, контроллер сам распределит их по строкам области. writePixels() пишет count пикселей из массива colors; writeColor() пишет один и тот же цвет count раз (быстро для заливки). writeData16() отправляет 16-битное значение старшим байтом вперёд (big-endian) — это естественный порядок ST7735 для RGB565.

5.5. Графические примитивы

Метод	Описание
drawPixel(x, y, color)	Один пиксель RGB565.
drawLine(x0, y0, x1, y1, color)	Отрезок. Диагонали — Liang-Barsky clips, не Bresenham на 65К итераций.
drawFastHLine(x, y, w, color)	Горизонтальный отрезок — одна SPI-транзакция.
drawFastVLine(x, y, h, color)	Вертикальный отрезок — одна SPI-транзакция.
drawRect(x, y, w, h, color)	Контур прямоугольника.
fillRect(x, y, w, h, color)	Залитый прямоугольник — через setAddrWindow + writeColor.
fillScreen(color)	Залить весь экран = fillRect(0, 0, _width, _height, color).
clear(color = BLACK)	Синоним fillScreen с дефолтом BLACK.

Внутренний drawLine использует алгоритм отсечения Liang-Barsky: если отрезок полностью или частично выходит за границы экрана, координаты cliраются до видимой части ДО запуска Bresenham. Это критично на AVR: без clips диагональ через весь экран генерировала бы ~65 000 итераций Bresenham с проверкой «за границей?» на каждой.

5.6. Окружности, скруглённые прямоугольники, эллипсы

```cpp
void drawCircle(x0, y0, r, color);
void fillCircle(x0, y0, r, color);
void drawRoundRect(x, y, w, h, r, color);
void fillRoundRect(x, y, w, h, r, color);
void drawEllipse(x0, y0, rx, ry, color);
void fillEllipse(x0, y0, rx, ry, color);
```

drawCircle / fillCircle — классический Midpoint-алгоритм. Округлённые прямоугольники (RoundRect) собираются из прямых линий и четвертинок окружности через приватные drawCircleHelper / fillCircleHelper. Эллипсы реализованы отдельным алгоритмом, потому что четвертинки окружности для прямоугольника с разными rx/ry не подходят.

fillArc (кольцевой сектор) компилируется только при ST7735_ENABLE_ARC=1 — он тянет sinf/cosf из libm.

```cpp
/* Доступно только при ST7735_ENABLE_ARC=1 */
void fillArc(x0, y0, rOut, rIn,
int16_t startDeg, int16_t endDeg, uint16_t color);
```

5.7. Треугольники

```cpp
void drawTriangle(x0, y0, x1, y1, x2, y2, color);
void fillTriangle(x0, y0, x1, y1, x2, y2, color);
```

fillTriangle реализован как Bresenham-scanline: для каждой Y-координаты от ymin до ymax вычисляется левая и правая граница скан-линии, затем рисуется горизонтальная линия drawFastHLine. Это быстрее, чем поточечная заливка.

5.8. Изображения RGB565 и 1-битные битмапы

Две категории методов: drawImage* для полноцветных RGB565-массивов и drawBitmap* для 1-битных масок (0 = прозрачный/фон, 1 = fg). Каждая категория имеет две версии — PROGMEM и RAM. На AVR массив ОБЯЗАН быть объявлен PROGMEM, иначе pgm_read пойдёт в flash и прочитает мусор; на ESP32/STM32 флеш отображена в адресное пространство, поэтому различия между PROGMEM и RAM стираются.

Метод	Тип данных
drawImage(x, y, w, h, const uint16_t *px)	RGB565, PROGMEM/flash
drawImageRAM(x, y, w, h, const uint16_t *px)	RGB565, RAM (работает на AVR)
drawBitmap(x, y, w, h, bitmap, fg)	1-бит, прозрачный фон, PROGMEM
drawBitmap(x, y, w, h, bitmap, fg, bg)	1-бит, непрозрачный фон, PROGMEM
drawBitmapRAM(x, y, w, h, bitmap, fg)	1-бит, прозрачный, RAM
drawBitmapRAM(x, y, w, h, bitmap, fg, bg)	1-бит, непрозрачный, RAM
drawBitmapScaled(x, y, w, h, bmp, fg, scale, invert=false)	1-бит с целочисленным масштабом и опц. инверсией

5.9. drawProgressBar — дифференциальная перерисовка

```cpp
void drawProgressBar(int16_t x, int16_t y, int16_t w, int16_t h,
int32_t value, int32_t prev,
int32_t vmax,
uint16_t fg, uint16_t bg, uint16_t border);
```

Прогресс-бар с дифференциальной перерисовкой: при вызове с новым value и предыдущим prev библиотека вычисляет только изменившуюся зону и перерисовывает её. Рамка и деления рисуются только в изменённой зоне — это устраняет мерцание. Если передать prev=-1, рисуется полная рамка и фон (первичная отрисовка).

5.10. Управление дисплеем

```cpp
void invertDisplay(bool on); /* INVON / INVOFF */
void setDisplayOn(bool on); /* DISPON / DISPOFF */
void sleep(); /* SLPIN — гаснет, GRAM сохраняется */
void wake(); /* SLPOUT + DISPON, блокирует 120 мс */
```

sleep() переводит дисплей в спящий режим: гаснет подсветка (если она управляется ST7735), но содержимое GRAM сохраняется. wake() выводит из сна: SLPOUT требует паузы ≥ 120 мс, библиотека её выдерживает сама. Используйте sleep/wake для экономии энергии в батарейных устройствах.

5.11. Аппаратный скролл

```cpp
void setScrollArea(int16_t topFixed,
int16_t scrollHeight,
int16_t botFixed); /* VSCRDEF */
void scrollTo(int16_t offset); /* VSCSAD */
```

ST7735 поддерживает вертикальный аппаратный скролл через две команды: VSCRDEF (0x33) задаёт три зоны — верхнюю фиксированную, прокручиваемую и нижнюю фиксированную; VSCSAD (0x37) указывает адрес в GRAM, который контроллер выведет как верхнюю строку прокручиваемой зоны. После scrollTo(offset) строки в прокручиваемой зоне «съезжают» аппаратно — CPU не копирует GRAM. Это основа быстрого терминала (см. §5.14 «Терминал»).

Ограничения HW-скролла на ST7735: поворот 0 (только портрет, некоторые табы не поддерживают HW-скролл при повороте), терминал должен занимать всю ширину экрана, высота терминала должна быть кратна lineHeight (целому числу строк).

5.12. Текст: шрифты, курсор, цвета, масштаб

```cpp
void setFont(const st7735_font_t *f);
const st7735_font_t* font() const;
void setTextScale(uint8_t s);
void setTextColor(uint16_t fg); /* прозрачный фон */
void setTextColor(uint16_t fg, uint16_t bg); /* непрозрачный */
void setTextWrap(bool w);
void setCursor(int16_t x, int16_t y);
int16_t cursorX() const;
int16_t cursorY() const;
uint16_t textWidth(const char *s) const;
uint16_t textWidth(const char *s, uint8_t scale) const;
uint16_t textHeight(uint8_t scale = 0) const; /* h*scale */
uint16_t lineHeight(uint8_t scale = 0) const; /* (h+1)*scale */
```

setFont() переключает активный шрифт (font5x7 или font7x11). setTextScale(s) задаёт целочисленный масштаб 1..N: каждый пиксель глифа превращается в блок s×s пикселей. textWidth возвращает ширину строки в пикселях (n×w×scale + (n−1)×scale — то есть учитывает однопиксельный пробел между глифами). textHeight = h×scale, lineHeight = (h+1)×scale — это шаг строки с учётом одной пустой строки пикселей между строками текста.

clearLine / clearLineBg — очистка строки

```cpp
void clearLine(int16_t x, int16_t y, int16_t w, uint8_t scale = 1);
void clearLine(int16_t x, int16_t y, int16_t w,
uint16_t bg, uint8_t scale);
```

Две перегрузки без часового значения bg. Раньше параметр bg=0xFFFF был часовым, что делало невозможным явный белый фон (0xFFFF молча превращалось в «фон не задан» → чёрный). Текущая схема: 4-аргументная версия берёт bg из текущего textBg (или BLACK, если textBg не задан), 5-аргументная — принимает bg явно и обязательно вместе с scale. Очистка делается через fillRect высотой lineHeight(scale).

5.13. Вывод: print, println, drawString, drawChar

print() имеет 8+ перегрузок для разных типов. На Arduino добавляются ещё две: для __FlashStringHelper (F("…")) и для String. На STM32 HAL эти две перегрузки недоступны — там нет Arduino WString.h.

Перегрузка	Назначение
print(const char *s)	Строка C.
print(char c)	Один символ (обёртка в char[2]).
print(int n)	Десятичное (через print(long)).
print(unsigned int n)	Десятичное беззнаковое (через unsigned long).
print(long n)	Десятичное знаковое.
print(unsigned long n)	Десятичное беззнаковое.
print(double d, uint8_t digits = 2)	С указанием числа знаков после точки (0..6).
print(int/long/uint/ulong n, uint8_t base)	С основанием 2..36 (BIN/OCT/DEC/HEX).
print(__FlashStringHelper *)	F("…") — flash-строка (только Arduino).
print(const String &s;)	Arduino WString (через c_str()).

println() имеет соответствующие перегрузки для каждого print() плюс безаргументная println() — она только переводит строку. printBase(long, base) — приватный, для знаковых в недесятичных основаниях (учитывает знак только в DEC, для BIN/HEX/OCT печатает как беззнаковое). printUnsignedBase(unsigned long, base) — для беззнаковых: 0x80000000UL в BIN даёт 32 цифры без

знака минус.

```cpp — текстовые методы
void drawString(const char *s, int16_t x, int16_t y,
uint16_t fg, uint16_t bg, uint8_t scale = 1);
void drawStringAligned(const char *s, int16_t x, int16_t y,
int16_t w, uint8_t align,
uint16_t fg, uint16_t bg, uint8_t scale = 1);
/* align: ST7735_ALIGN_LEFT / CENTER / RIGHT */

void drawChangedString(int16_t x, int16_t y,
const char *new_str, const char *prev_str,
uint16_t fg, uint16_t bg, uint8_t scale = 1);

void putStr5x7(uint8_t scale, uint16_t x, uint16_t y,
const char *str, uint16_t color, uint16_t bgcolor);

uint16_t drawChar(int16_t x, int16_t y, uint16_t code,
uint16_t fg, uint16_t bg, uint8_t scale); /* непрозр. */
uint16_t drawChar(int16_t x, int16_t y, uint16_t code,
uint16_t fg, uint8_t scale); /* прозрачный */
```

drawStringAligned() выравнивает строку по ширине w — самый важный метод для безмерцательных виджетов. Он сам заливает поля слева/справа цветом bg, поэтому каждый пиксель в области [x, x+w] пишется ровно один раз. drawChangedString() сравнивает new_str и prev_str посимвольно и перерисовывает только изменившиеся глифы — экономия SPI для статического текста, который редко меняется.

drawChar(code, …) принимает код глифа в текущей кодировке шрифта (обычно CP1251 после UTF-8→CP1251 преобразования). Возвращает ширину отрисованного глифа в пикселях (w*scale + scale для пробела).

5.14. Терминал (только при ST7735_ENABLE_TERMINAL=1)

```cpp
void setTerminal(int16_t x, int16_t y, int16_t w, int16_t h);
void setTerminal(); /* выключить терминал */
void setTerminalBg(uint16_t c); /* сменить фон + перерисовать */
void termClear();
bool terminalOn() const;
```

Терминал — это прямоугольная область экрана с автоскроллом. После setTerminal(x, y, w, h) вызовы print()/println() пишут не в курсор, а в эту область: текст идёт по строкам длиной ST7735_TERM_COLS, при заполнении строки — перенос на следующую, при заполнении всех ST7735_TERM_ROWS строк — сдвиг вверх на одну строку. setTerminalBg() меняет цвет фона и сразу перекрашивает область + перерисовывает буфер. termClear() очищает буфер и экран.

Аппаратный скролл терминала

Терминал поддерживает аппаратный скролл через VSCRDEF/VSCSAD: вместо копирования строк в RAM (медленно и чревато UB через memmove) контроллеру задаётся смещение, и он сам «сдвигает» изображение. Это даёт ~95% экономии SPI при лог-выводе по сравнению с программным скроллом. Аппаратный скролл включается вызовом:

```cpp — аппаратный скролл терминала
/* Включить аппаратный скролл терминала (VSCRDEF/VSCSAD). */
void termHwScroll(bool on);
bool termHwScrollOn() const; /* текущее состояние */
```

Ограничения HW-скролла: поворот 0 (портрет), терминал должен занимать всю ширину экрана, высота _termH должна быть кратна lineHeight (целому числу строк). Если условия не выполнены, termHwScroll(true) молча переключается в программный режим — скролл работает, но медленнее.

5.15. printf (опционально)

```cpp
/* Доступно только при ST7735_ENABLE_PRINTF=1 */
void printf(const char *fmt, ...)
__attribute__((format(printf, 2, 3)));
```

printf компилируется только при ST7735_ENABLE_PRINTF=1 и тянет vsnprintf из libc (~200 Б + 1–2 КБ, если больше никто его не зовёт). Спецификатор %f требует отдельного флага линкера (-Wl,-u,vfprintf -lprintf_flt) для libprintf_flt на AVR — без него %f не работает. Используйте ST7735Value::set() вместо printf для числового вывода: там форматирование сделано через inline st7735_fmtDouble() без зависимостей.

6. UI-виджеты (st7735ui.h)

Модуль st7735ui.h — header-only: все классы определены прямо в заголовке. Каждый класс компилируется только если соответствующий макрос ST7735_ENABLE_* задан в 1 перед #include. В этой главе — детальный справочник по всем восьми классам: конструктор, методы, пример.

6.1. ST7735Button — кнопка с press-состоянием

```cpp
ST7735Button(ST7735 &tft;, int16_t x, int16_t y, int16_t w, int16_t h,
const char *label,
uint16_t fg, uint16_t bg,
uint16_t border, uint16_t pressBg, uint8_t scale = 1);

void draw();
void setPressed(bool p);
bool isPressed() const;
bool contains(int16_t x, int16_t y) const;
```

Кнопка рисуется как скруглённый прямоугольник (fillRoundRect + drawRoundRect с радиусом 3). При setPressed(true) заливка меняется на pressBg, при setPressed(false) — обратно на bg. Подпись центрируется по высоте через drawStringAligned(ST7735_ALIGN_CENTER). Вся кнопка рисуется в одной SPI-сессии (startWrite/endWrite). Состояние хранится в поле _pressed; если новое состояние равно старому, setPressed молча возвращается без перерисовки.

```cpp — пример кнопки
ST7735Button btn(tft, 5, 5, 60, 24, "OK",
ST7735_WHITE, ST7735_BLUE,
ST7735_WHITE, ST7735_RED, 1);
btn.draw();
/* при касании: */
btn.setPressed(true);
/* при отпускании: */
btn.setPressed(false);
```

6.2. ST7735Toggle — тумблер ON/OFF

```cpp
ST7735Toggle(ST7735 &tft;, int16_t x, int16_t y, int16_t w, int16_t h,
uint16_t onColor, uint16_t offColor,
uint16_t knobColor, uint16_t borderColor);

void draw();
void setOn(bool on);
bool isOn() const;
void toggle();
bool contains(int16_t x, int16_t y) const;
```

Тумблер — скруглённый прямоугольник с двумя состояниями: knob (кружок) слева при off и справа при on. Заливка корпуса меняется между offColor и onColor. Радиус скругления равен половине высоты, что даёт «капсулу». Оптимизация: при setOn() заливается только внутренность (на 1 px внутрь), рамка не затирается — иначе после первого переключения рамка пропадала бы (был такой баг в ранних версиях).

cpp — пример тумблера

```
ST7735Toggle sw(tft, 5, 5, 40, 18,
ST7735_GREEN, ST7735_DARKGRAY,
ST7735_WHITE, ST7735_WHITE);
sw.draw();
sw.setOn(true); /* теперь ON */
```

6.3. ST7735Checkbox — чекбокс с подписью

cpp

```
ST7735Checkbox(ST7735 &tft;, int16_t x, int16_t y, int16_t size,
uint16_t borderColor, uint16_t checkColor,
uint16_t bgColor, const char *label,
uint16_t textColor, uint8_t scale = 1);

void draw();
void setChecked(bool c);
bool isChecked() const;
void toggle();
bool contains(int16_t x, int16_t y) const;
```

Чекбокс — квадрат size×size с рамкой и опциональной подписью справа (через 4 px). При setChecked(true) внутри квадрата рисуется галочка двумя линиями: (x+2, mid) → (x+2*size/5, size-3) → (x+size-3, y+2). Область нажатия contains() учитывает фактическую позицию текста: если шрифт крупнее квадрата, текст выходит выше _y, и область нажатия расширяется до min(_y, ty) сверху и max(_y+size, ty+th) снизу.

cpp — пример чекбокса

```
ST7735Checkbox cb(tft, 5, 5, 16,
ST7735_WHITE, ST7735_GREEN, ST7735_BLACK,
"Включить", ST7735_YELLOW, 1);
cb.draw();
cb.setChecked(true);
```

6.4. ST7735Value — числовой индикатор (главный виджет)

Главный виджет для вывода чисел: напряжение, ток, температура, давление. Фиксированная зона (x, y, w), внутри — одна строка. Форматирование через st7735_fmtDouble() (inline в st7735.h) — без dtostrf, без snprintf, без math.h. Единица измерения вынесена в параметр _unit (по умолчанию "V", можно задать "mV", "mA", "°C", "%", "" и т.д.).

```cpp — ST7735Value API
ST7735Value(ST7735 &tft;, int16_t x, int16_t y, int16_t w,
uint8_t align, uint16_t fg, uint16_t bg,
uint8_t scale = 2,
const char *unit = "V", bool autoScale = true);

void set(double v, uint8_t digits = 0xFF);
void setError(const char *msg, uint16_t errFg = ST7735_RED);
void setRaw(const char *s, bool force = false);
void setUnit(const char *unit);
void setTextColor(uint16_t fg);
void setTextColor(uint16_t fg, uint16_t bg);
void setAutoScale(bool enable);
void invalidate();
void draw();
const char *text() const;
const char *unit() const;
bool autoScale() const;
```

Skip-redraw оптимизация

При вызове set(v, digits) виджет форматирует число в строку через st7735_fmtDouble(), и сравнивает результат с ранее отображённой строкой. Если отформатированная строка не изменилась — SPI-вывод пропускается полностью. Для polling-датчиков, где значение колеблется вокруг одной величины (например, 25.0 °C ± 0.05), экономия SPI достигает 99%: 99 из 100 вызовов set() не делают ничего, кроме сравнения строк.

Авто-масштаб

Если _autoScale=true и при текущем масштабе текст не влезает в _w (textWidth > _w), виджет понижает effScale на 1 и пробует снова, пока не влезет или effScale > 1. Это полезно при смене единицы измерения (V → mV): "1234.5 mV" при scale=2 шире, чем "12.34 V", авто-масштаб снизит до scale=1 автоматически. Поле _prevScale хранит последний применённый масштаб — при его смене зона над новым текстом дочищается fillRect'ом (хвост от более крупного предыдущего значения).

Состояния виджета

Внутренний enum ValueType { VT_NONE, VT_NUMBER, VT_ERROR, VT_RAW } определяет тип текущего содержимого _cur. Это нужно для setUnit(): если последнее set() было числом (VT_NUMBER), при смене unit виджет переформатирует и перерисует. Если VT_ERROR или VT_RAW — unit обновится в поле, но экран не перерисуется (строка остаётся как есть, например "READY").

```cpp — пример ST7735Value
ST7735Value volt(tft, 5, 50, 80, ST7735_ALIGN_RIGHT,
ST7735_GREEN, ST7735_BLACK, 2, "V");

volt.set(12.345, 3); /* "12.345 V" */
volt.set(0.5, 2); /* "0.50 V" — fillRect стирает "12.345" */
volt.setError("ERR"); /* "ERR" красным */
volt.set(99.9, 1); /* "99.9 V" */
volt.setUnit("mA");
volt.set(1234.5, 1); /* "1234.5 mA" — авто-масштаб → scale=1 */
```

6.5. ST7735BarGauge — шкальный индикатор с порогом

cpp — ST7735BarGauge API

```
ST7735BarGauge(ST7735 &tft;, int16_t x, int16_t y, int16_t w, int16_t h,
double minVal = 0.0, double maxVal = 30.0,
uint16_t barColor = ST7735_GREEN,
uint16_t bgColor = ST7735_BLACK,
uint16_t borderColor = ST7735_WHITE,
uint16_t warnColor = ST7735_RED,
double warnThreshold = 28.0);

void draw();
void setValue(double val);
void setRange(double minVal, double maxVal,
double warnThreshold = 1e30);
void setTicks(uint8_t ticks);
```

Горизонтальная полоса с рамкой, делениями шкалы и предупреждающим цветом. Дифференциальная перерисовка: при setValue(val) виджет вычисляет новую длину fillPix = (val-min)/(max-min) × innerW, сравнивает с _prevFillPix и рисует только изменившуюся зону. Если длина не изменилась и цвет не изменился — выход без SPI. При смене цвета (val перешёл порог warnThreshold) заливается вся длина fillPix новым цветом, без предварительной очистки — новый цвет непрозрачен.

drawTicks(from, to) — приватный, рисует вертикальные деления шкалы только в диапазоне X-координат [from, to]. Это та часть дифф-перерисовки, которая восстанавливает риски после заливки полосы: если fillPix изменился с 10 на 20, риски перерисовываются только в колонках [10, 20], а не во всём innerW.

cpp — пример BarGauge

```
ST7735BarGauge bar(tft, 5, 70, 118, 16,
0.0, 30.0,
ST7735_GREEN, ST7735_BLACK, ST7735_WHITE,
ST7735_RED, 28.0);
bar.draw();
bar.setValue(15.0);
bar.setValue(29.5); /* выше 28 → красный */
bar.setRange(0.0, 100.0, 80.0); /* смена диапазона на лету */
```

6.6. ST7735Plotter — осциллограф/график реального времени

```cpp — ST7735Plotter API
ST7735Plotter(ST7735 &tft;, int16_t x, int16_t y, int16_t w, int16_t h,
double minVal = 0.0, double maxVal = 30.0,
uint16_t lineColor = ST7735_YELLOW,
uint16_t bgColor = ST7735_BLACK,
uint16_t gridColor = 0x2104,
uint16_t borderColor = ST7735_WHITE);

void show(); void draw(); /* draw = show (compat) */
void hide();
bool isVisible() const;
void addValue(double val);
void clear();
void setRange(double minVal, double maxVal);
void setAutoRange(bool enable);
void resetPeakHold();
void invalidate();

bool autoRange() const;
double effMinVal() const;
double effMaxVal() const;
double peakMin() const;
double peakMax() const;
double peakToPeak() const;
```

График реального времени с кольцевым буфером _rawMv[ST7735_PLOTTER_MAX_PTS] значений в милливольтах (всего 240 Б RAM при MAX_PTS=120). Каждая addValue округляет значение до мВ, кладёт в буфер по индексу _head, сдвигает _head на 1 (с возвратом по модулю maxCap), обновляет пиковые значения и (если _autoRange) эффективный диапазон. После данных действий вызывается redrawGraph() — если окно видимо.

Multi-strip рендер

Если ширина графика innerW > ST7735_PLOTTER_MAX_W (дефолт 160), график рисуется несколькими полосами по ST7735_PLOTTER_MAX_W пикселей. Без multi-strip больший график обрезался бы справа — теперь он рисуется полностью, ценой нескольких setAddrWindow вместо одного.

Инкрементальный рендер

При ST7735_PLOTTER_INCREMENTAL=1 (дефолт на не-AVR) перерисовываются только изменившиеся колонки, не всё внутреннее поле. Каждая addValue добавляет одну точку → меняются сегменты линии только в затронутых колонках. Экономия SPI до 95% для медленных сигналов. На AVR макрос выключен по умолчанию (дефолт 0) — там проще один раз перерисовать всё, чем вести буфер «что изменилось».

Оверлей ВЫНЕСЕН из класса

Ранние версии Plotter рисовали оверлей (AUTO, P-P, Mx) через putStr5x7 прямо поверх графика — это давало мерцание в углу. В v5.0 оверлей вынесен из класса: рисуйте его отдельными ST773Value над/под окном графика, используя геттеры peakMin() / peakMax() / peakToPeak() / effMinVal() / effMaxVal(). Это убирает мерцание и сокращает код Plotter (ушли strcpy/strncat/st7735_fmtDouble/putStr5x7 из класса).

cpp — пример Plotter с оверлеем

```
ST7735Plotter plot(tft, 5, 50, 118, 60,
0.0, 30.0,
ST7735_YELLOW, ST7735_BLACK,
0x2104, ST7735_WHITE);

/* Внешний оверлей пиков */
ST7735Value peakMax(tft, 5, 30, 58, ST7735_ALIGN_LEFT,
ST7735_MAGENTA, ST7735_BLACK, 1, "Mx");
ST7735Value peakMin(tft, 65, 30, 58, ST7735_ALIGN_RIGHT,
ST7735_CYAN, ST7735_BLACK, 1, "Mn");

void loop() {
double v = 15.0 + 10.0 * sin(t * 0.05);
plot.addValue(v);
peakMax.set(plot.peakMax(), 2);
peakMin.set(plot.peakMin(), 2);
}
```

6.7. ST7735Log — журнал сообщений

cpp — ST7735Log API

```
ST7735Log(ST7735 &tft;, int16_t x, int16_t y, int16_t w, int16_t h,
uint16_t fg = ST7735_WHITE,
uint16_t bg = ST7735_BLACK,
uint8_t scale = 1);

void add(const char *s, uint16_t fg = 0);
void clear();
void show();
void hide();
bool isVisible() const;
uint16_t total() const;
```

Кольцевой буфер из ST7735_LOG_LINES строк по ST7735_LOG_COLS символов (по умолчанию 8×30, RAM = 264 Б). Каждой строке можно задать свой цвет через параметр fg в add(); fg=0 означает «цвет по умолчанию» (для чёрного текста на белом используйте 0x0001, т.к. 0 — это сигнальное значение). add() обрезает строку до ST7735_LOG_COLS, не разрывая UTF-8 (откатывается назад, если конец попадает на continuation-байт). При visible, add() либо дорисовывает одну строку (если буфер ещё не скроллится), либо перерисовывает все видимые строки непрозрачно (drawStringAligned заливает свою ячейку).

cpp — пример журнала

```
ST7735Log log(tft, 5, 70, 118, 60);
log.show();
log.add("System ready", ST7735_GREEN);
log.add("Sensor init...", ST7735_WHITE);
log.add("ERROR: timeout", ST7735_RED);
```

6.8. ST7735Screens — переключатель экранов

cpp — ST7735Screens API

```
typedef void (*ST7735ScreenFn)(void *ctx);

ST7735Screens(ST7735 &tft;, uint16_t bg = ST7735_BLACK);

int8_t add(ST7735ScreenFn onEnter,
ST7735ScreenFn onUpdate = 0,
ST7735ScreenFn onLeave = 0,
void *ctx = 0);

void select(uint8_t i);
void begin(uint8_t first = 0);
void next();
void prev();
void update();

uint8_t current() const;
uint8_t count() const;
bool is(uint8_t i) const;
```

Циклический переключатель экранов: main → график → логи → main … Каждый экран — три функции-колбэка (без std::function, работает на AVR): onEnter(ctx) — нарисовать статику (рамки, подписи), вызвать show() виджетов; onUpdate(ctx) — периодическое обновление из loop(); onLeave(ctx) — hide() виджетов (сбор данных продолжается). При переключении экран один раз заливается bg, затем зовётся onEnter.

cpp — пример переключателя экранов

```
void onEnterMain(void *) { /* рисуем статику */ }
void onUpdateMain(void *) { /* обновляем значения */ }

void onEnterPlot(void *) { plot.show(); }
void onUpdatePlot(void *) { plot.addValue(readADC()); }
void onLeavePlot(void *) { plot.hide(); }

ST7735Screens scr(tft);
scr.add(onEnterMain, onUpdateMain); /* экран 0: main */
scr.add(onEnterPlot, onUpdatePlot, onLeavePlot); /* экран 1: plot */
scr.begin(0);

void loop() {
scr.update();
if (button_pressed) scr.next(); /* по кнопке — след. экран */
}
```

7. Шрифты

7.1. font5x7 и font7x11

Библиотека поставляется с двумя шрифтами, оба покрывают латиницу, кириллицу (CP1251, включая белорусские Ўў Іі) и базовые символы. Шрифты хранятся в массивах PROGMEM и описываются структурой st7735_font_t с полями data, w, h, bpc, flags, first/count, first2/count2.

Шрифт	w	h	Формат	Описание
font5x7	5	7 (h=8 в дескрипторе)	column-major, bpc=1	5×7 — классический glcd-шрифт. h=8 в дескрипторе из-за descender'ов g/p/q/y.
font7x11	7	11 (h=11)	row-major, flags bit0=1	7×11 — более читаемый, row-major для скорости на AVR.

7.2. Дескриптор шрифта

```cpp
typedef struct st7735_font_t {
const uint8_t *data; /* байты глифов */
uint8_t w; /* ширина клетки, px */
uint8_t h; /* высота клетки, px */
uint8_t bpc; /* байт на колонку (col-major); =1 для row-major */
uint8_t flags; /* бит0: 1 = row-major (7x11) */
uint16_t first; /* первый код диапазона 1 */
uint16_t count; /* число глифов диапазона 1 */
uint16_t first2; /* первый код диапазона 2 (0 = нет) */
uint16_t count2; /* число глифов диапазона 2 */
} st7735_font_t;
```

Два диапазона (first/count + first2/count2) нужны, чтобы не хранить пустые блоки между ASCII (0x20..0x7E) и кириллицей CP1251 (0xC0..0xFF). Бит flags bit0 определяет формат хранения: 0 = column-major (классический, bpc=h/8 округлённое вверх), 1 = row-major (bpc=1, построчно). Для font7x11 row-major быстрее на AVR: одна строка пикселей — один байт.

7.3. Кириллица UTF-8 → CP1251

Шрифты хранят кириллицу в кодировке CP1251. Метод print(const char*) автоматически преобразует UTF-8 в CP1251 на лету: распознаёт многобайтовые последовательности (0xD0..0xD1 для русского, 0xD0 0x8E / 0xD0 0x8E для белорусских Ўў, 0xD0 0x86 / 0xD0 0x87 для Іі) и переводит в однобайтовый индекс. Это работает для русских, украинских (Єє Її) и белорусских (Ўў Іі) букв.

```cpp
tft.setCursor(5, 55);
tft.print("Привет, мир!"); /* UTF-8 → CP1251 автоматически */
tft.print(" Беларусь: Ўў Іі"); /* белорусские буквы тоже работают */
```

7.4. Формулы размеров текста

textWidth, textHeight и lineHeight рассчитываются по простым формулам. Знание формул полезно при расчёте размеров виджетов: нужно заранее оценить, влезет ли значение в зону при данном шрифте и масштабе.

Функция	Формула	Замечание
textWidth(s, scale)	n × w × scale + (n − 1) × scale	n — число символов; между глифами однопиксельный пробел, тоже масштабируется.
textHeight(scale)	h × scale	h из дескриптора шрифта (8 для font5x7, 11 для font7x11).
lineHeight(scale)	(h + 1) × scale	Шаг строки с учётом одной пустой строки пикселей.

7.5. Памятка: сколько символов влезает

При портрете 128 px шириной и виджете _w=118 px число влезающих символов зависит от шрифта и масштаба. Шаг глифа (pitch) = (w+1)×scale. Таблица ниже — справочная: выбирая scale, прикидывайте, влезет ли ожидаемая строка (например, "12.345 V" = 8 символов) в ширину виджета.

Шрифт	scale	pitch (px)	Max символов в 118 px
font5x7	1	6	19
font5x7	2	12	9
font7x11	1	8	14
font7x11	2	16	7

8. Цвета (color565.h)

8.1. Формат RGB565

ST7735 хранит каждый пиксель в GRAM как 16-битное значение RGB565: 5 бит на красный, 6 бит на зелёный, 5 бит на синий. Зелёный получает лишний бит, потому что человеческий глаз более чувствителен к зелёному диапазону. Макрос RGB565(R, G, B) преобразует 24-битный RGB в 16-битный формат, отбрасывая младшие биты: ((R>>3)<<11) | ((G>>2)<<5) | (B>>3).

```cpp
inline uint16_t RGB565(uint8_t R, uint8_t G, uint8_t B) {
return ((R >> 3) << 11) | ((G >> 2) << 5) | (B >> 3);
}

uint16_t myColor = RGB565(100, 200, 50); /* салатовый */
```

8.2. Предопределённые константы

color565.h определяет базовый набор цветов в RGB565. В st7735.h они доступны также с префиксом ST7735_ (например, ST7735_RED = COLOR565_RED).

Константа	RGB565	RGB(8-bit)
COLOR565_BLACK	0x0000	0, 0, 0
COLOR565_WHITE	0xFFFF	255, 255, 255
COLOR565_RED	0xF800	255, 0, 0
COLOR565_LIME / GREEN	0x07E0	0, 255, 0 — ярко-зелёный
COLOR565_BLUE	0x001F	0, 0, 255
COLOR565_CYAN	0x07FF	0, 255, 255
COLOR565_MAGENTA	0xF81F	255, 0, 255
COLOR565_YELLOW	0xFFE0	255, 255, 0
COLOR565_ORANGE	0xFD20	255, 165, 0
COLOR565_PINK	0xFE19	255, 192, 203
COLOR565_PURPLE	0x780F	128, 0, 128
COLOR565_GRAY	0x8410	128, 128, 128 — средний серый
COLOR565_DARKGRAY	0x4208	64, 64, 64
COLOR565_NAVY	0x000F	0, 0, 128
COLOR565_MAROON	0x7800	128, 0, 0
COLOR565_OLIVE	0x7BE0	128, 128, 0
COLOR565_TEAL	0x0410	0, 128, 128

9. Оптимизации

Эта глава — для тех, кто хочет понять, почему драйвер быстрый. Все оптимизации направлены на одну цель: минимизировать количество пикселей, записываемых в GRAM за один кадр обновления. Запись в GRAM — это SPI-трафик, а SPI на AVR не бесконечен. Принцип безмерцательного вывода: каждый пиксель пишется ровно один раз.

9.1. Таблица оптимизаций

#	Оптимизация	Эффект	Условие
1	Plotter инкрементальный рендер	Перерисовываются только изменившиеся колонки (diff по _rawMv). Экономия SPI до 95% для медленных сигналов.	не-AVR (ST7735_PLOTTER_INCREMENTAL=1)
2	ST7735Value skip-redraw	Сравнение по отформатированной строке. Если не изменилась — SPI пропускается. Экономия до 99% для polling-датчиков.	все платформы
3	Терминал HW scroll	VSCRDEF/VSCSAD — контроллер сам сдвигает строки. ~95% экономии при лог-выводе.	ST7735_ENABLE_TERMINAL=1, поворот 0
4	drawLine Liang-Barsky clips	Диагонали вне экрана cliраются до запуска Bresenham. Нет 65 000 пустых итераций.	все платформы
5	writePixelsBytes	Прямая запись байт-массива RGB565 без промежуточного uint16_t col[] буфера. Минус копирование.	внутренний метод Plotter/изображений
6	Multi-strip Plotter	График шире ST7735_PLOTTER_MAX_W рисуется несколькими полосами. Не обрезается справа.	все платформы
7	memmove в termScroll	Убран UB при копировании перекрывающихся буферов. В десятки раз быстрее при больших размерах терминала.	ST7735_ENABLE_TERMINAL=1
8	st7735_fmtDouble epsilon	Округление HALF-AWAY-FROM-ZERO с epsilon = 0.5 / 10^digits. Защита от IEEE-754 неточности (нет «:» вместо цифры).	inline в st7735.h, все платформы

9.2. Принцип безмерцательного вывода

Каждый пиксель в обновляемой зоне должен быть записан ровно один раз за кадр. Классический анти-паттерн «залить фон → стереть → нарисовать значение» пишет каждый пиксель дважды или трижды, что и даёт мерцание (пиксель успевает стать фоновым, потом снова значением, глаз это ловит). Драйвер вместо этого:

- Вычисляет изменившуюся зону до SPI-вывода и пишет только её.
- Для текста использует drawStringAligned(), который заливает поля слева/справа цветом bg одновременно с глифами — каждый пиксель один раз.
- Для BarGauge хранит _prevFillPix и пишет только дельту между ним и новым fillPix.
- Для Plotter предвычисляет lo[cx]/hi[cx] — диапазон строк, занятых линией в колонке cx — и для каждой (r, cx) решает цвет одним сравнением, без двойной записи.

9.3. Стратегия: batch через startWrite/endWrite

Каждая SPI-транзакция имеет накладные расходы: опустить CS, отправить CASET+RASET+RAMWR (9 байт), потом данные, потом поднять CS. На AVR это ~50 мкс на транзакцию. Если виджет рисует 100 точек по одной — 5 мс уходит на накладные. Решение: батчить операции внутри одной startWrite() / endWrite(). Вложенные startWrite() считаются через _inWrite, и CS не поднимается до закрытия всех уровней. Все виджеты (Button, Toggle, Checkbox, BarGauge, Plotter, Log) оборачивают свою отрисовку в одну startWrite/endWrite сессию.

9.4. Кэш окна setAddrWindow

Поле _wx0/_wx1/_wy0/_wy1 хранит последнее открытое окно GRAM. Если следующий setAddrWindow попадает в то же окно — повторная команда CASET+RASET не отправляется, пишется только RAMWR. Это экономит 8 байт SPI на каждой операции в пределах одного окна. Кэш инвалидируется при setRotation, setOffsets, startWrite (если другой клиент SPI менял окно).

10. Платформы

10.1. AVR (ATmega328PB / MiniCore)

Самая ограниченная платформа: 2 КБ RAM, 32 КБ flash, double=float. Драйвер здесь работает, но требует осознанного выбора макросов: включайте только нужные виджеты, уменьшайте ST7735_CHUNK до 32, если не хватает стека, выключайте ST7735_PLOTTER_INCREMENTAL (дефолт 0). ST7735_ENABLE_PRINTF требует флаг линкера -Wl,-u,vfprintf -lprintf_flt — без него %f не работает. Используйте ST7735Value::set() вместо printf — там форматирование сделано inline st7735_fmtDouble().

10.2. ESP32 / ESP8266

Можно включить все макросы безболезненно: ≥ 320 КБ RAM, много flash. ST7735_PLOTTER_INCREMENTAL=1 (дефолт) — инкрементальный рендер Plotter включён. SPI можно поднять до 40 МГц (ESP32 HSPI/VSPI). На ESP8266 flash отображена в RAM через ICACHE, PROGMEM не нужен (но не мешает).

10.3. STM32duino

Аналогично ESP32: все макросы включаются свободно. SPI через HardwareSPI библиотеки STM32duino. Пины — любые GPIO с поддержкой SPI alternate function (например, BluePill: PA4-PA7 для SPI1). ST7735_PLOTTER_INCREMENTAL=1.

10.4. STM32 HAL (CubeMX)

Через макрос ST7735_BARE_STM32 (определяется автоматически, если ARDUINO не задан). Пины задаются макросами ST7735_CS_GPIO / ST7735_CS_PIN / ST7735_DC_GPIO / ST7735_DC_PIN / ST7735_RST_GPIO / ST7735_RST_PIN. По умолчанию берутся CS=PA3, DC=PA4, RST=PA2 — старая схема автора. SPI-инстанс через ST7735_SPI_HANDLE (по умолчанию &hspi1;). Все операции делаются через HAL_GPIO_WritePin и HAL_SPI_Transmit.

cpp — STM32 HAL

```cpp
/* main.h или -D флаги компилятора */
#define ST7735_CS_GPIO GPIOA
#define ST7735_CS_PIN GPIO_PIN_3
#define ST7735_DC_GPIO GPIOA
#define ST7735_DC_PIN GPIO_PIN_4
#define ST7735_RST_GPIO GPIOA
#define ST7735_RST_PIN GPIO_PIN_2
#define ST7735_SPI_HANDLE (&hspi1;)

/* Код приложения */
#include "st7735.h"
ST7735 tft(-1, -1, -1, ST7735R_BLACK); /* пины из макросов */
tft.begin(8000000);
```

11. Примеры

Пять рабочих примеров поставляются в examples/. Каждый — это самостоятельный .ino-скетч, готовый к загрузке в Arduino IDE. Ниже — ключевые фрагменты с пояснениями.

11.1. Hello.ino — минимальная проверка

Заливка экрана, текст, примитивы (drawRect, fillCircle, drawEllipse, drawLine), кириллица UTF-8 → CP1251, число с плавающей точкой через print(double, digits). Без UI-виджетов — чистый драйвер.

cpp — Hello.ino

```cpp
ST7735 tft(10, 9, 8, ST7735R_BLACK);

void setup() {
Serial.begin(9600);
tft.begin();
tft.setRotation(0);
tft.fillScreen(ST7735_BLACK);

tft.setTextColor(ST7735_YELLOW, ST7735_BLACK);
tft.setFont(&font7x11;);
tft.setTextScale(2);
tft.setCursor(5, 5);
tft.print("Hello!");

tft.setTextColor(ST7735_WHITE, ST7735_BLACK);
tft.setTextScale(1);
tft.setCursor(5, 35);
tft.print("ST7735 driver");

tft.setCursor(5, 55);
tft.print("Привет, мир!"); /* UTF-8 → CP1251 */

tft.drawRect(5, 80, 60, 40, ST7735_GREEN);
tft.fillRect(10, 85, 50, 30, ST7735_BLUE);
tft.drawCircle(95, 100, 20, ST7735_RED);
tft.fillCircle(95, 100, 15, ST7735_MAGENTA);
tft.drawEllipse(64, 100, 30, 20, ST7735_ORANGE);

tft.setCursor(5, 155);
tft.setTextColor(ST7735_CYAN, ST7735_BLACK);
tft.print("pi=");
tft.print(3.14159, 4); /* pi=3.1416 */
}

void loop() {}
```

11.2. Volt.ino — ST7735Value

Числовой индикатор напряжения с автопереформатированием. Демонстрирует: set(double, digits), setError(), setUnit(), авто-масштаб при смене единицы измерения с V на mA.

cpp — Volt.ino

```cpp
#include "st7735.h"
#define ST7735_ENABLE_VALUE 1
#include "st7735ui.h"

ST7735 tft(10, 9, 8, ST7735R_BLACK);
ST7735Value volt(tft, 5, 50, 80, ST7735_ALIGN_RIGHT,
ST7735_GREEN, ST7735_BLACK, 2, "V");

void setup() {
Serial.begin(9600);
tft.begin();
tft.setRotation(0);
tft.fillScreen(ST7735_BLACK);
tft.setFont(&font7x11;);

volt.set(12.345, 3); /* "12.345 V" */
delay(1500);
volt.set(0.5, 2); /* "0.50 V" */
delay(1500);
volt.setError("ERR"); /* "ERR" красным */
delay(1500);
volt.set(99.9, 1); /* "99.9 V" */
delay(1500);

volt.setUnit("mA");
volt.set(1234.5, 1); /* "1234.5 mA" — авто-масштаб → scale=1 */
}

void loop() {
static double v = 0.0;
static int8_t dir = 1;
v += dir * 0.137;
if (v > 25.0) { v = 25.0; dir = -1; }
if (v < -5.0) { v = -5.0; dir = 1; }
volt.set(v, 2);
delay(100);
}
```

11.3. BarGauge.ino — ST7735BarGauge

Шкальный индикатор 0..30V с порогом 28V. При превышении порога полоса становится красной. Демонстрирует setRange() — смену диапазона на лету.

cpp — BarGauge.ino

```cpp
#include "st7735.h"
#define ST7735_ENABLE_BARGAUGE 1
#include "st7735ui.h"

ST7735 tft(10, 9, 8, ST7735R_BLACK);
ST7735BarGauge bar(tft, 5, 70, 118, 16,
0.0, 30.0,
ST7735_GREEN, ST7735_BLACK, ST7735_WHITE,
ST7735_RED, 28.0);

void setup() {
Serial.begin(9600);
tft.begin();
tft.setRotation(0);
tft.fillScreen(ST7735_BLACK);

bar.draw();
bar.setValue(5.0); delay(500);
bar.setValue(15.0); delay(500);
bar.setValue(25.0); delay(500);
bar.setValue(29.5); /* выше 28 → красный */
delay(1000);

bar.setRange(0.0, 100.0, 80.0); /* смена диапазона */
bar.setValue(95.0); /* выше 80 → красный */
}

void loop() {
static double v = 0.0;
static int8_t dir = 1;
v += dir * 0.3;
if (v > 100.0) { v = 100.0; dir = -1; }
if (v < 0.0) { v = 0.0; dir = 1; }
bar.setValue(v);
delay(50);
}
```

11.4. Plotter.ino — ST7735Plotter с внешним оверлеем

График-осциллограф 118×60px в центре экрана. Синусоида 0.5 Гц + медленный дрейф. Внешний оверлей пиков (peakMax, peakMin) через ST7735Value над графиком — рисуется отдельными виджетами через геттеры plot.peakMax() / plot.peakMin().

cpp — Plotter.ino

```cpp
#include
#include "st7735.h"
#define ST7735_ENABLE_VALUE 1
#define ST7735_ENABLE_PLOTTER 1
#include "st7735ui.h"

ST7735 tft(10, 9, 8, ST7735R_BLACK);

ST7735Plotter plot(tft, 5, 50, 118, 60,
0.0, 30.0,
ST7735_YELLOW, ST7735_BLACK,
0x2104, ST7735_WHITE);

ST7735Value peakMax(tft, 5, 30, 58, ST7735_ALIGN_LEFT,
ST7735_MAGENTA, ST7735_BLACK, 1, "Mx");
ST7735Value peakMin(tft, 65, 30, 58, ST7735_ALIGN_RIGHT,
ST7735_CYAN, ST7735_BLACK, 1, "Mn");

void setup() {
Serial.begin(9600);
tft.begin();
tft.setRotation(0);
tft.fillScreen(ST7735_BLACK);
tft.setFont(&font7x11;);

tft.drawStringAligned("OSCILLOSCOPE", 5, 5, 118,
ST7735_ALIGN_CENTER,
ST7735_WHITE, ST7735_BLACK, 1);
plot.draw();
peakMax.set(0.0, 2);
peakMin.set(0.0, 2);
}

void loop() {
static uint32_t t = 0;
double v = 15.0 + 10.0 * sin(t * 0.05) + 3.0 * sin(t * 0.013);
plot.addValue(v);
peakMax.set(plot.peakMax(), 2);
peakMin.set(plot.peakMin(), 2);
t++;
delay(50);
}
```

11.5. UI_Demo.ino — полный демо

Полный демо всех виджетов на одной портретной раскладке 128×160: Checkbox, четыре ST7735Value, BarGauge и Plotter. Демонстрирует также setError(), setRaw(), set(INFINITY) (что должно дать "OVF"), смену единицы измерения °C ↔ °F и сбор пиков в Serial через геттеры.

cpp — UI_Demo.ino

```cpp
#include
#include "st7735.h"

#define ST7735_ENABLE_BUTTON 0
#define ST7735_ENABLE_TOGGLE 0
#define ST7735_ENABLE_CHECKBOX 1
#define ST7735_ENABLE_VALUE 1
#define ST7735_ENABLE_BARGAUGE 1
#define ST7735_ENABLE_PLOTTER 1
#include "st7735ui.h"

ST7735 tft(10, 9, 8, ST7735R_BLACK);

ST7735Checkbox cb(tft, 5, 5, 16, ST7735_WHITE, ST7735_GREEN,
ST7735_BLACK, "Option", ST7735_YELLOW, 1);
ST7735Value volt(tft, 5, 25, 118, ST7735_ALIGN_RIGHT,
ST7735_GREEN, ST7735_BLACK, 2, "V");
ST7735Value temp(tft, 5, 47, 118, ST7735_ALIGN_RIGHT,
ST7735_YELLOW, ST7735_BLACK, 2, "\xB0""C");
ST7735Value v(tft, 5, 91, 118, ST7735_ALIGN_RIGHT,
ST7735_CYAN, ST7735_BLACK, 2, "V");
ST7735BarGauge bar(tft, 5, 115, 118, 12,
0.0, 30.0,
ST7735_GREEN, ST7735_BLACK, ST7735_WHITE,
ST7735_RED, 28.0);
ST7735Plotter plot(tft, 5, 130, 118, 28,
0.0, 30.0,
ST7735_YELLOW, ST7735_BLACK,
0x2104, ST7735_WHITE);

void setup() {
Serial.begin(9600);
tft.begin();
tft.setRotation(0);
tft.fillScreen(ST7735_BLACK);
tft.setFont(&font7x11;);

cb.draw(); cb.setChecked(true);
volt.set(12.345, 2);
temp.set(25.5, 1);
v.set(1.234, 3);
bar.draw();
plot.draw();
for (uint8_t i = 0; i < 120; i++) {
plot.addValue(15.0 + 10.0 * sin(i * 0.1));
delay(150);
}
Serial.print("P-P="); Serial.println(plot.peakToPeak());
}

void loop() {}
```

12. Troubleshooting / FAQ

Подборка типичных проблем и решений. Если ваша проблема здесь не описана — проверьте issues на GitHub и создайте новое, приложив минимальный воспроизводимый скетч.

Проблема: Белый экран при подаче питания

Решение: Проверьте таб (ST7735R_BLACK vs ST7735R_RED) и питание 3.3 В. Подача 5 В на VCC дисплея — частая причина. Также проверьте, что пин 10 (SS AVR) сконфигурирован как OUTPUT: если он INPUT и плавает, SPI переходит в slave-режим.

Проблема: Цвета инвертированы (красный ↔ зелёный)

Решение: Поменяйте таб в конструкторе. ST7735R_RED и ST7735R_BLACK различаются битом BGR/RGB в MADCTL. Если цвет плёнки на стекле красный — берите ST7735R_RED, если чёрная — ST7735R_BLACK.

Проблема: Текст обрезается справа

Решение: Уменьшите setTextScale(s) или включите авто-масштаб в ST7735Value (autoScale=true по умолчанию). Также проверьте, что зона виджета _w достаточна: при font7x11 scale=2 шаг глифа = 16 px, в 80 px влезает 5 символов (см. §7.5 «Памятка»).

Проблема: 'ST7735Value' does not name a type

Решение: Добавьте #define ST7735_ENABLE_VALUE 1 перед #include "st7735ui.h". Аналогично для других виджетов: каждый класс требует своего макроса. Это сознательное поведение — вы включаете только то, что используете.

Проблема: Мерцание при обновлении значения

Решение: Используйте ST7735Value::set() вместо ручного «fillRect → setCursor → print». ST7735Value рисует через drawStringAligned, который заливает поля и глифы одним SPI-потоком — каждый пиксель пишется ровно один раз.

Проблема: График обрезается справа

Решение: Увеличьте ST7735_PLOTTER_MAX_W (например до 320 для 320×240 экрана) перед #include. Multi-strip рендер справится, но больший MAX_W ускоряет — меньше setAddrWindow.

Проблема: RAM не хватает на AVR (freeRAM < 200)

Решение: Выключите неиспользуемые ST7735_ENABLE_* макросы. Уменьшите ST7735_CHUNK до 32 (это 64 Б стека вместо 128). Уменьшите ST7735_TERM_ROWS / ST7735_LOG_LINES. На ATmega328PB 2 КБ RAM — это предел, дальше только ESP32 / STM32.

Проблема: _freeRAM() < 200 после включения всех виджетов

Решение: Включайте только нужные. См. §4.2 «Принцип включай только нужное». Полный набор всех 8 виджетов на AVR не влезает.

Проблема: Ошибки при компиляции с Adafruit_GFX

Решение: ST7735 не зависит от Adafruit_GFX. Файл Adafruit_ST7735_GFX.h — опциональный мост. Если вы используете только API ядра, Adafruit_GFX не нужен. Если мост нужен — подключайте его #include "Adafruit_ST7735_GFX.h" ПОСЛЕ st7735.h, и задайте ST7735_ENABLE_Adafruit 1.

Проблема: printf с %f не работает на AVR

Решение: ST7735_ENABLE_PRINTF=1 тянет vsnprintf, но %f требует флага линкера -Wl,-u,vfprintf -lprintf_flt. Добавьте его в platform.txt или .platformio.ini. Или используйте ST7735Value::set() — там форматирование сделано через inline st7735_fmtDouble() без зависимостей.

Проблема: Кириллица отображается вопросиками или мусором

Решение: Убедитесь, что исходный .ino-файл сохранён в UTF-8. Arduino IDE сохраняет в UTF-8 по умолчанию. Если редактор — Visual Studio Code или другой — проверьте кодировку файла. Драйвер сам преобразует UTF-8 → CP1251 при выводе.

Проблема: Белорусские буквы Ўў Іі не выводятся

Решение: Эти буквы поддерживаются в UTF-8 → CP1251 преобразовании. Убедитесь, что в .ino-файле они записаны как UTF-8 (байты 0xD0 0x8E / 0xD0 0x8E для Ў, 0xD0 0x86 / 0xD0 0x87 для І). Если редактор подставил однобайтовую кодировку — драйвер их не распознает.

13. Лицензия и контакты

13.1. Лицензия

Библиотека ST7735 Arduino Driver распространяется свободно для любого использования: личных проектов, коммерческих изделий, образовательных целей. Attribution (указание авторства) приветствуется, но не обязателен. Код поставляется «как есть», без гарантий работоспособности в конкретных условиях — проверяйте на стенде перед использованием в серийных изделиях.

13.2. Контакты и исходный код

Исходный код библиотеки, примеры и история изменений публикуются на GitHub. Issues, Pull Requests и обсуждения приветствуются.

Поле	Значение
GitHub	https://github.com/AlecsHome
Версия доки	v5.0
Платформы	AVR / ESP32 / STM32duino / STM32 HAL
Язык	C++11 (Arduino IDE и bare STM32)
Зависимости	SPI.h (Arduino) или HAL_SPI_Transmit (STM32)

Конец документа